

HW1 Singer classification

R13942143 何景盛

Code&Checkpoint: https://drive.google.com/drive/folders/1DA3wHZgbm5fHhAqTLtKw-yPGWx0cJEj9?usp=drive_link

Outline

- Task 1: Train a Traditional Machine Learning Model
- Task 2: Train a Deep Learning Model

Outline

- Task 1: Train a Traditional Machine Learning Model
- Task 2: Train a Deep Learning Model

Preprocessing/Model

- **Preprocessing**

- Each song is segmented into 15-second clips (no overlap)
- Extract MFCC features for each clip
 - n_mfcc: 20
 - n_fft: 2048
 - N_mels: 40
 - Hop_length: 512

- **Model (Classifier)**

- Xgboost
 - n_estimators: 300
 - learning_rate: 0.05
 - max_depth: 6
 - subsample: 0.8
 - colsample_bytree: 0.8,

Training

1. Split each song into 15-second clips (no overlap)
2. Extract MFCC features for each clip
3. Compute mean and standard deviation of MFCCs and concatenate into MFCC vector
4. Use MFCC vectors with corresponding labels as input to XGBoost for classification training

Inference

1. Split each song into 15-second clips with 50% overlap
2. Extract MFCC features for each clip
3. Compute mean and standard deviation of MFCCs and concatenate into MFCC vector
4. Feed MFCC vectors into the trained XGBoost classifier
5. Average the predicted probabilities of all clips and obtain song-level probabilities for 20 classes
6. Choose the class with the highest probability as Top-1, and the top three classes as Top-3

Why clip 15 second?

- Conducted experiments with different clip lengths for training and inference
- Accuracy varies depending on clip length
- Best performance achieved when training and inference are both at 15 seconds

Val/train (acc)	5 (s)	10 (s)	15 (s)	20 (s)	30 (s)
5 (s)	0.6104	0.6104	0.6104	0.5887	0.5498
10 (s)	0.5801	0.6234	0.6277	0.5974	0.5887
15 (s)	0.5844	0.6017	0.6494	0.632	0.5931
20 (s)	0.5671	0.6147	0.5974	0.6017	0.6104

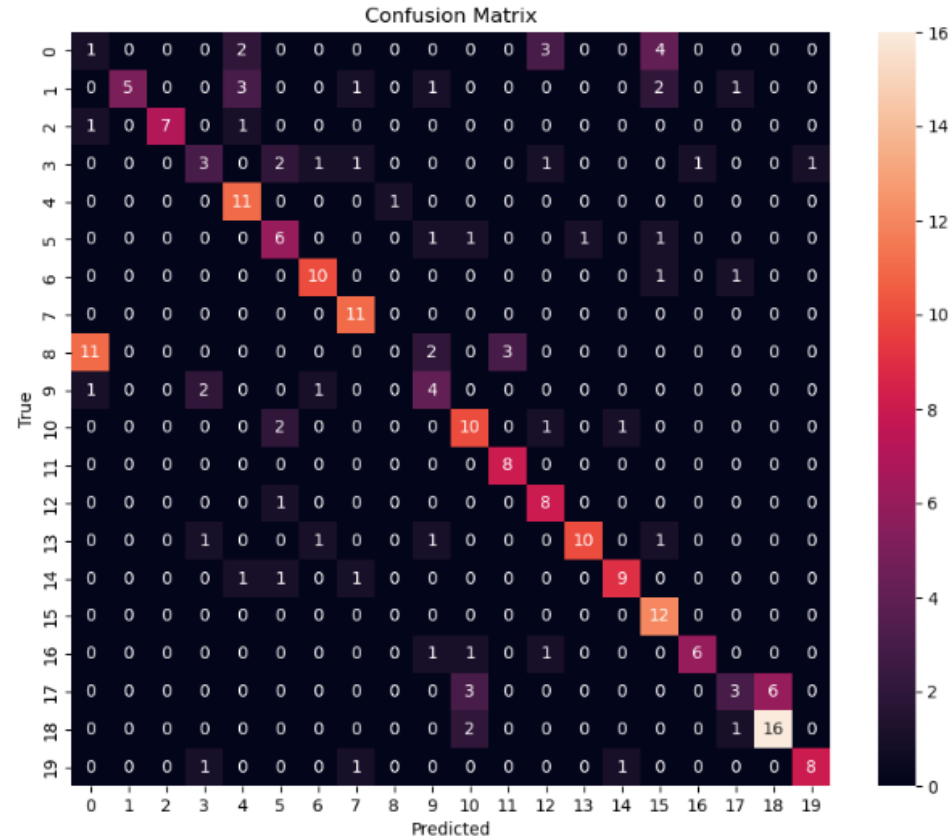
Train data
clip time



val data
clip time

Validation results

- Accuracy :
 - Top1: 0.6407
 - Top3: 0.8095



Outline

- Task 1: Train a Traditional Machine Learning Model
- Task 2: Train a Deep Learning Model

Preprocessing

- Split each song into 15-second clips (no overlap)
- Apply Demucs (mdx_extra) for source separation and extract vocals from each clip

Dataloader

- Apply data augmentation on vocals clip refer to reference:
 - RandomApply([PolarityInversion()], p=0.8),
 - RandomApply([Noise(min_snr=0.3, max_snr=0.5)], p=0.3),
 - RandomApply([Gain()], p=0.2),
 - RandomApply([HighLowPass(sample_rate=16000)], p=0.8),
 - RandomApply([Delay(sample_rate=16000)], p=0.5),
 - RandomApply([PitchShift(n_samples=15*16000, sample_rate=16000)], p=0.4),
 - RandomApply([Reverb(sample_rate=16000)], p=0.3),
- Extract Mel-spectrogram features from each augmented vocal clip:
 - N_fft: 2048
 - N_mels: 128
 - Hop_length: 512

Model

- ShortChunkCNN (without beginning Mel-spectrogram layer, because it has already applied in dataloader)
- Other hyperparameters are the same as the reference

Training

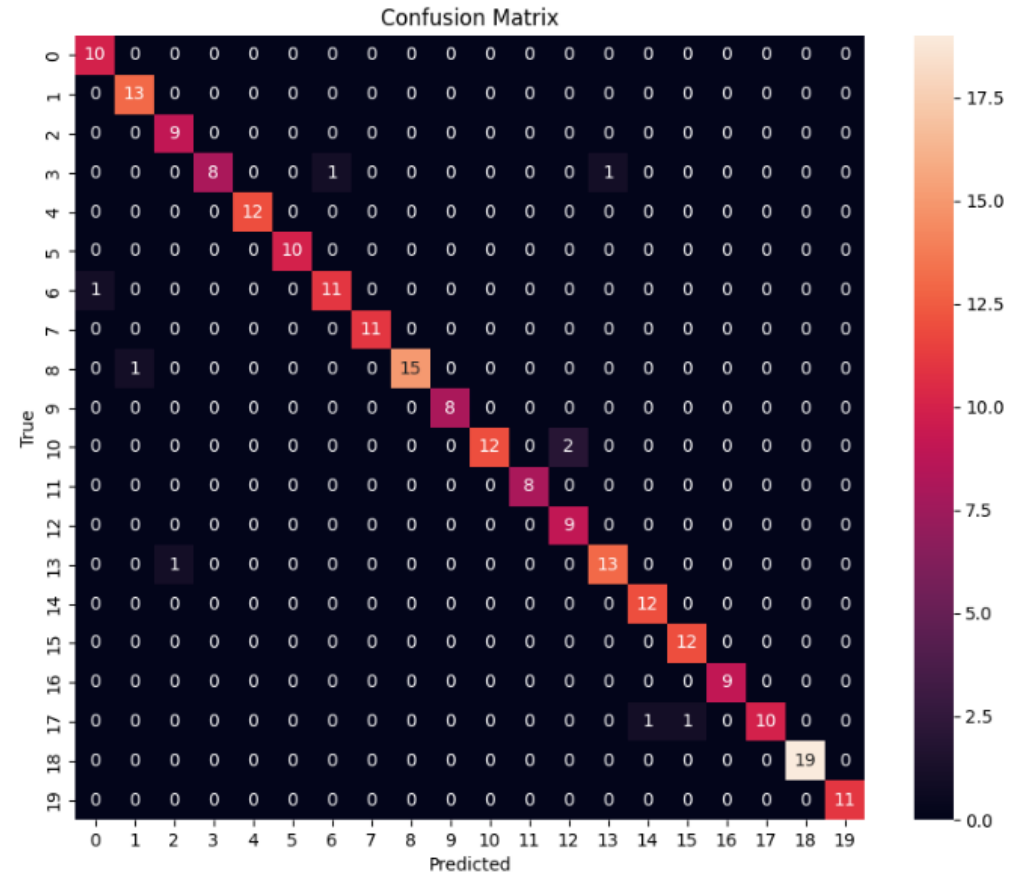
1. Split each song into 15-second clips (no overlap)
2. Apply Demucs (mdx_extra) for source separation and extract vocals from each clip
3. Apply data augmentation
4. Compute Mel-spectrogram for each augmented vocal clip
5. Feed features and corresponding labels into ShortChunkCNN and using CrossEntropyLoss for training

Inference

1. Split each song into 15-second clips with 50% overlap
2. Apply Demucs (mdx_extra) for source separation and extract vocals from each clip
3. Compute Mel-spectrogram for each vocal clip
4. Feed Mel-spectrograms into the trained ShortChunkCNN for classification
5. Average prediction probabilities across all clips and obtain song-level probabilities for 20 classes
6. Top-1 class: class with highest probability; Top-3 classes: three highest probabilities

Validation results

- Accuracy :
 - Top1: 0.9610
 - Top3: 0.9740



References

1. [PyTorch tutorial — Music Classification: Beyond Supervised Learning, Towards Real-world Applications](#)
2. <https://github.com/minzwon/sota-music-tagging-models/blob/master/training/model.py>
3. <https://github.com/facebookresearch/demucs>